



Harmely

App de découverte musicale

React Native ■ Firebase ■ API Spotify

Auteurs

HONORINE Kylian
GRONDIN Angélique

Cadre

SAE — RT2 TP1

☰ Sommaire

1	Présentation	1
1.1	Stack technique	1
2	Mise en place de l'environnement	1
3	Authentification (AuthScreen.js)	1
4	Navigation principale (Navigation.js)	2
5	Système de swipe (SwipeScreen.js)	2
6	Chat temps réel (ChatScreen.js)	2
7	Recherche musicale (SearchScreen.js)	3
8	Historique : Liked / Disliked	3
9	Intégration Spotify (SpotifyService.js)	3
10	Configuration Firebase (firebaseConfig.js)	3
11	Récapitulatif des dépendances	4
12	Difficultés rencontrées	4
13	Conclusion	4

1 Présentation

📣 Pitch produit

Harmely est une application mobile de découverte musicale. Inspirée du modèle **Tinder pour la musique**, elle propose à l'utilisateur des extraits Spotify via un système de swipe — droite pour aimer, gauche pour passer — et conserve l'historique des préférences. Un chat en temps réel permet de partager ses découvertes.

1.1 Stack technique

Couche	Technologie
Front mobile	React Native (via Expo)
Backend	Firebase Authentication + Firestore
Persistence	AsyncStorage (local) + Firestore (cloud)
API tierce	Spotify Web API (extraits 30 s)
Navigation	React Navigation (stack)
Temps réel	Firestore onSnapshot

2 Mise en place de l'environnement

1. Installation de **Node.js** et **npm** (gestion des paquets).
2. Initialisation du projet : **npm init** génère un **package.json**.
3. Installation d'Expo + CLI :

```
npm install -g expo-cli
expo init harmely
cd harmely
npm install firebase react-native expo-av expo-font expo-splash-screen
```

3 Authentification (AuthScreen.js)

Firebase Authentication gère la connexion et l'inscription. Les messages d'erreur sont traduits en français pour améliorer l'UX. Persistance via **AsyncStorage**.

```
npm install @react-native-async-storage/async-storage
npm install @expo/vector-icons expo-linear-gradient
```

Fonctions clés

- **handleLogin** — connexion via email/mot de passe
- **handleSignUp** — création de compte et redirection
- **translateError** — mapping Firebase → français
- **loadFonts + prepare** — splash screen et polices personnalisées

4 Navigation principale (Navigation.js)

Une navigation **stack** empile les écrans (**AuthScreen**, **SwipeScreen**, **ChatScreen**, etc.). **react-native-screens** optimise les transitions natives.

```
npm install @react-navigation/native @react-navigation/stack \
  react-native-screens react-native-safe-area-context \
  react-native-gesture-handler react-native-reanimated react-native-svg
```

5 Système de swipe (SwipeScreen.js)

Cœur de l'app : **PanResponder** détecte les gestes de glissement et déclenche des animations selon la direction. **expo-av** joue les extraits Spotify.

👉 Logique de geste

- Swipe droite ($\geq$ seuil) → ajout aux `likedTracks`
- Swipe gauche ($\leq$ -seuil) → ajout aux `dislikedTracks`
- Geste insuffisant → animation de retour à la position initiale

Dépendances :

- `react-native-gesture-handler` — gestion des swipes
- `react-native-deck-swiper` — composant deck-like
- `expo-av` — lecture audio des extraits

6 Chat temps réel (ChatScreen.js)

Firestore synchronise les messages en temps réel via `onSnapshot`. Chaque conversation est identifiée par un `chatId`, et chaque message inclut l'identité de l'expéditeur et un timestamp.

```
import { onSnapshot, collection, addDoc, query, orderBy } from "firebase/firestore";

const q = query(
  collection(db, "chats", chatId, "messages"),
  orderBy("timestamp", "asc")
);
const unsubscribe = onSnapshot(q, (snapshot) => {
  setMessages(snapshot.docs.map(d => ({ id: d.id, ...d.data() })));
});
```

7 Recherche musicale (SearchScreen.js)

Une fonction de normalisation des chaînes (`normalizeString`) standardise la requête utilisateur (accents, casse). Les résultats sont triés par pertinence et popularité, limités aux 20 premiers, filtrés pour n'inclure que les morceaux avec extrait audio disponible.

Dépendances : `expo-network` (vérifie la connexion avant requête) et `buffer` (encodage des données API).

8 Historique : Liked / Disliked

Deux écrans miroirs (`LikedTracksScreen.js`, `DislikedTracksScreen.js`) affichent les morceaux via `FlatList`. Chaque ligne montre titre, artiste, album — avec `"inconnu"` en cas de données manquantes.

9 Intégration Spotify (SpotifyService.js)

Service centralisé pour toutes les requêtes Spotify. L'authentification se fait via **client credentials** : `clientId` + `clientSecret` configurés dans la console Spotify Developer.

```

async function getAccessToken() {
  const credentials = base64.encode(`${clientId}:${clientSecret}`);
  const res = await fetch("https://accounts.spotify.com/api/token", {
    method: "POST",
    headers: {
      "Authorization": `Basic ${credentials}`,
      "Content-Type": "application/x-www-form-urlencoded"
    },
    body: "grant_type=client_credentials"
  });
  const data = await res.json();
  return data.access_token;
}

```

Deux fonctions principales :

- `getRandomTracksByGenre(genre)` — alimente le swipe (défaut : « hip-hop » + « french »)
- `searchTracks(query)` — alimente la recherche utilisateur

Les deux filtrent pour ne retenir que les morceaux disposant d'`preview_url` non null.

10 Configuration Firebase (`firebaseConfig.js`)

`initializeAuth` avec persistance `AsyncStorage` garantit que la session survit aux redémarrages de l'app. `getCurrentUser` fournit aux écrans l'identité de l'utilisateur connecté pour personnaliser l'expérience.

11 Récapitulatif des dépendances

Package	Rôle
<code>expo</code>	SDK React Native
<code>firebase</code>	Auth + Firestore
<code>@react-navigation/*</code>	Navigation entre écrans
<code>react-native-gesture-handler</code>	Gestion des swipes
<code>react-native-deck-swiper</code>	Composant deck
<code>expo-av</code>	Lecture audio des previews
<code>expo-linear-gradient</code>	Dégradés UI
<code>expo-font, expo-splash-screen</code>	Polices, splash screen
<code>@react-native-async-storage/async-storage</code>	Persistance locale
<code>spotify-web-api-node</code>	Client Spotify
<code>axios</code>	Requêtes HTTP
<code>react-native-base64</code>	Encodage clés API

12 Difficultés rencontrées

Problème	Solution apportée
react-native-reanimated exige une config native supplémentaire	Suivi de la doc officielle, configuration Babel
Hétérogénéité des machines de l'équipe	Synchronisation via GitHub, sessions partagées
Gestion des tokens Spotify (expiration, scopes)	Itérations dans Spotify Developer Console, refresh à la demande

13 Conclusion

Harmely combine des briques modernes du développement mobile : framework cross-platform, backend serverless, API tierce, gestes naturels. La SAE a permis de couvrir le cycle complet d'une app : conception UX, intégration API, persistance, temps réel, et déploiement.

Au-delà du livrable, l'expérience confirme une réalité du métier : l'essentiel du travail consiste à **intégrer** des services existants et à soigner l'expérience utilisateur — pas à réécrire l'OAuth ou la lecture audio.

« La meilleure interface est celle qu'on ne voit pas. »